Week 2 Progress Report

Network Quality Monitoring System - Automatic Checks, Threshold Alerting & Summaries

Prepared by	Zohaib Akmal
Report date	1 November 2026
Plan period (Week 2)	26 October - 1 November 2026
Completed on	1 November 2026 (on schedule)
Repository	github.com/L1S24BSSE0071/Network-Quality-Monitoring-System
Version tag & tests	week-2 tagged; 193 automated pytest tests passing

1. Summary

The Week 2 milestone and all planned Week 2 deliverables are complete: enabled HTTP targets are checked automatically in the background at their own intervals, every check outcome is evaluated against configurable warning and critical thresholds, state-change events (warning, critical, recovered) are logged without creating noisy duplicates on repeated failures, and new API endpoints expose event logs, time-windowed check history, and per-target uptime and latency summaries. All work is merged to **main**, tagged **week-2**, and verified by **193 automated pytest tests**.

2. Milestone

"A test target is taken offline and a critical event appears in the log, followed by a recovery event when it returns." Status: **Met**.

3. Tasks against the plan

Planned task	Status	Output
Mon - Tue: Add the scheduler so each target runs at its own interval and results are stored with timestamps	**Done**	app/scheduler.py: APScheduler BackgroundScheduler with ThreadPoolExecutor(20), immediate initial check, and live sync on create/edit/delete
Wed: Buffer day to finish unfinished work; optional ping check if time allows	**Done**	Caught up from Week 1 delay; hardened concurrency, SQLite QueuePool sizing, and mid-check URL edit handling (HTTP kept as core check)
Wed - Thu: Handle timeouts, DNS failures, invalid URLs and certificate errors without crashing the scheduler	**Done**	app/checker.py: _describe_connect_error cause-chain inspection, daemon worker deadline for hung OS DNS, exception isolation in run_scheduled_check
Thu - Fri: Evaluate warning and critical thresholds and create events on state changes, including recovery	**Done**	app/evaluator.py: evaluate_state, state_from_last_severity, evaluate_transition, and atomic record_state_change inside run_check_and_store
Fri - Sat: Calculate uptime percentage and build endpoints for history, events and summaries	**Done**	calculate_uptime_pct; GET /targets/summaries, GET /targets/{id}/summary, GET /targets/{id}/results?hours=..., GET /targets/{id}/events, GET /events
Sun 1 Nov: Write unit tests for threshold logic and prepare the Week 2 review	**Done**	tests/test_evaluator.py (66 tests), tests/test_scheduler.py (17 tests); docs/week2-review.md; docs/week2-progress-report.pdf; tag week-2

4. Deliverables

Deliverable	Location in repository
Background scheduler	app/scheduler.py
Threshold evaluator & uptime calculator	app/evaluator.py
History, event log & summary API endpoints	app/routers/targets.py, app/schemas.py, app/main.py
Updated system design & README	docs/design.md, README.md
Automated test suite (193 tests)	tests/test_api.py, tests/test_checker.py, tests/test_evaluator.py, tests/test_scheduler.py
Week 2 review notes & PDF report	docs/week2-review.md, docs/week2-progress-report.pdf

5. Definition of done

Criterion	Result	Evidence
Scheduled checks run for at least two targets without blocking the API	**Met**	ThreadPoolExecutor(20) runs checks concurrently; DB connections are released before network I/O
Taking a test target offline produces exactly one critical event and bringing it back produces one recovery event	**Met**	Verified in test_offline_then_online_milestone_scenario and test_full_state_lifecycle_creates_events_only_on_changes
The threshold unit tests pass	**Met**	66 unit and integration tests in tests/test_evaluator.py (193 total tests passing in ~6 s)
Work is committed and tagged week-2	**Met**	All Week 2 feature, fix and docs branches merged (--no-ff) into main; tag week-2 pushed

6. Implementation, debugging and optimization

State-transition model (app/evaluator.py): Each check outcome is classified as *normal*, *warning* (latency_ms > warning_ms), or *critical* (down, missing/non-finite latency, or latency_ms > critical_ms). The target's previous state is inferred from its latest stored event (none or *recovered* -> normal; *warning* -> warning; *critical* -> critical). An Event row is inserted atomically inside the check's BEGIN IMMEDIATE transaction only when the state changes, including Critical -> Warning (degraded improvement) and Warning/Critical -> Normal (recovered).

Uptime, history & summary endpoints (app/routers/targets.py): Added GET /targets/summaries and GET /targets/{id}/summary returning current status (normal, warning, critical, or unknown before the first check), latest check details, total/up/failed check counts, uptime_pct (rounded to 2 decimal places), and avg_latency_ms, with optional ?hours=1..720 window filtering. Added GET /targets/{id}/events and GET /events (with severity and target_id filters), plus ?hours=... filtering on GET /targets/{id}/results.

Performance & concurrency optimizations: (1) Combined the post-check Target verification and latest Event.severity lookup inside run_check_and_store into a single pre-built query (_POST_CHECK_TARGET_QUERY), halving SELECT queries inside the BEGIN IMMEDIATE write lock. (2) Pre-built parameterized SQLAlchemy select() and delete() statements with bindparam() across all modules (2.46x faster query execution). (3) Replaced full 9-column Target ORM hydration prior to manual/scheduled checks with a 2-column tuple query. (4) Used a covering-index correlated subquery on ix_check_results_target_checked to join each target's latest check result by primary key in O(log M).

Test suite module	Coverage	Result
tests/test_api.py	75 tests: Target CRUD, IDNA/IPv6/URL/port/credential validation, control chars, strict types, cascade delete, concurrent PATCH serialization	Passed

Test suite module	Coverage	Result
tests/test_checker.py	35 tests: 200 OK, redirects, 404/503, headers-only streaming, timeout, hung DNS abandonment, DNS/TLS/refused error mapping, stale check rejection	Passed
tests/test_evaluator.py	66 tests: Threshold boundaries, all 9 state transitions, 500-char truncation, non-finite/bool latency guards, mid-check threshold edits, uptime math, /events & /summaries endpoints	Passed
tests/test_scheduler.py	17 tests: Immediate check, per-target intervals, skipping disabled/ping targets, live sync, mid-check URL edit retry, failure isolation, 8-thread concurrent edits	Passed

Total automated test suite: 193 passed in ~6 seconds (zero network access, isolated temporary SQLite DB).

7. Demonstration evidence

Target goes offline & recovers - `GET /targets/1/events`


```
[{"id": 2, "target_id": 1, "occurred_at": "2026-11-01T05:18:10Z",
  "severity": "recovered", "latency_ms": 42.0,
  "message": "Target recovered (latency 42 ms, threshold 500 ms)"},
 {"id": 1, "target_id": 1, "occurred_at": "2026-11-01T05:17:10Z",
  "severity": "critical", "latency_ms": null,
  "message": "Target is down: Connection refused"}]
```


Target uptime & latency summary - `GET /targets/1/summary`


```
{"target_id": 1, "name": "Example", "host_or_url": "https://example.com",
 "check_type": "http", "interval_seconds": 60, "warning_ms": 500,
 "critical_ms": 1000, "enabled": true, "status": "normal",
 "last_checked_at": "2026-11-01T05:18:10Z", "last_is_up": true,
 "last_latency_ms": 42.0, "last_http_status": 200, "last_error_message": null,
 "total_checks": 4, "up_checks": 3, "failed_checks": 1,
 "uptime_pct": 75.0, "avg_latency_ms": 45.2}
```


8. Problems encountered and resolutions

Problem	Resolution
Python's :g float format truncated 7-digit latencies (10000.25 -> 10000.2) and used scientific notation above 1,000,000 ms.	Added _format_ms() in app/evaluator.py to format latencies up to 2 decimal places without trailing zeros or scientific notation.
Modifying host_or_url while a scheduled check was in flight could cause APScheduler (max_instances=1) to skip the immediate wakeup.	Added a bounded retry loop in run_scheduled_check so the active worker immediately checks the new URL after discarding the stale check.
Separate Target and Event.severity queries inside BEGIN IMMEDIATE doubled read round-trips under the write lock.	Combined both into a single pre-built query (_POST_CHECK_TARGET_QUERY) with a correlated scalar subquery (_LAST_SEVERITY_SUBQ).

9. Plan for Week 3 (2 - 8 November 2026: Dashboard)

- Build the web dashboard layout (HTML, CSS, JavaScript) served by FastAPI and connected to /targets/summaries, /targets/{id}/results, and /events.
- Show per-target status indicators (normal, warning, critical, unknown), last check time, latest latency, and uptime summary cards/table.
- Add the Chart.js latency history chart with a selectable time-range filter (?hours=...).
- Add forms to create, edit, enable/disable, and delete targets from the browser with clear validation messages.
- Add status filtering, the event log view, in-app alert banner, and CSV report export.